

Relatório Técnico Síntese

Integrantes: Gabriel Dario da Rosa

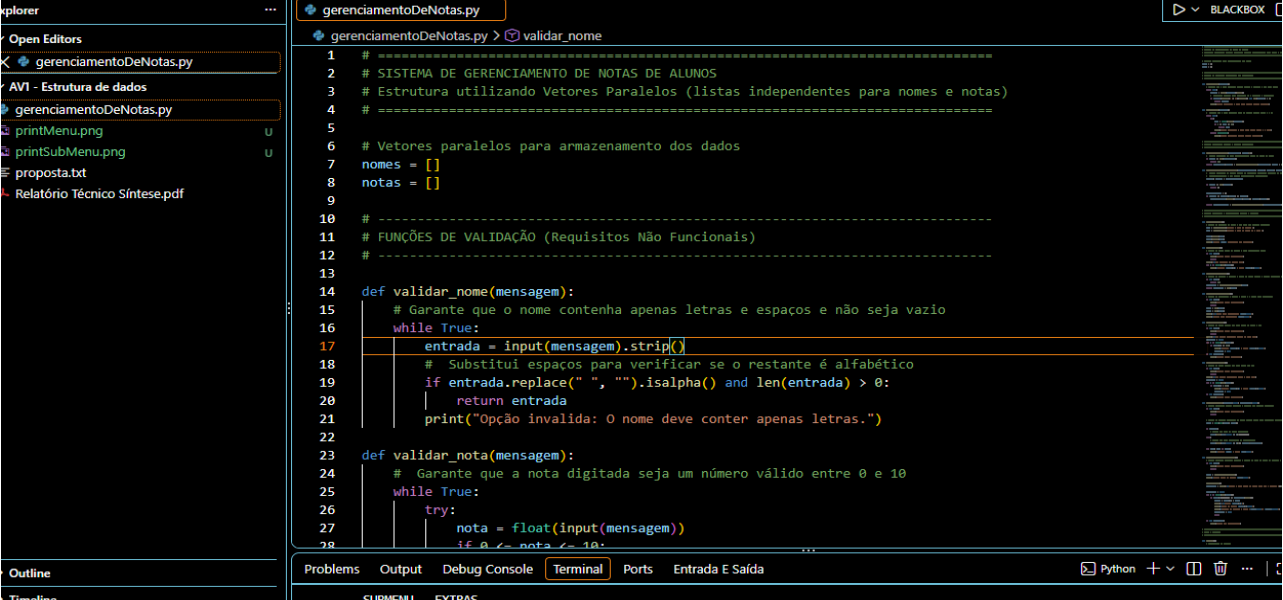
Visão Geral do Projeto

O objetivo deste projeto foi desenvolver uma aplicação via terminal em **C** e **Python** voltada para o gerenciamento completo de alunos e suas respectivas notas. A aplicação combina um menu principal com operações fundamentais de manipulação de dados e um submenu focado em funcionalidades avançadas de análise estatística e ordenação.

Recursos e Funcionalidades Implementadas

- **Menu Principal:** Permite cadastro de dados, listagem geral, cálculo da média da turma, identificação das notas extremas (maior e menor) e exibição rápida dos alunos aprovados (≥ 7.0).
- **Submenu Estatístico e De Ordenação:**
 1. **Ordenação por Nome:** Listagem da turma em ordem alfabética.
 2. **Classificação por Desempenho:** Ordenação das notas em ordem crescente e decrescente.
 3. **Filtragem por Critérios:** Filtro para alunos reprovados (< 7.0) e busca por intervalo de notas ($[x, y]$).
 4. **Análise de Desvio da Média:** Cálculo do desvio individual de cada aluno em relação à média geral da turma.
- **Requisitos Não Funcionais:** Validação rigorosa de entradas de dados (bloqueio de caracteres inválidos em nomes e notas fora de formato/faixa).

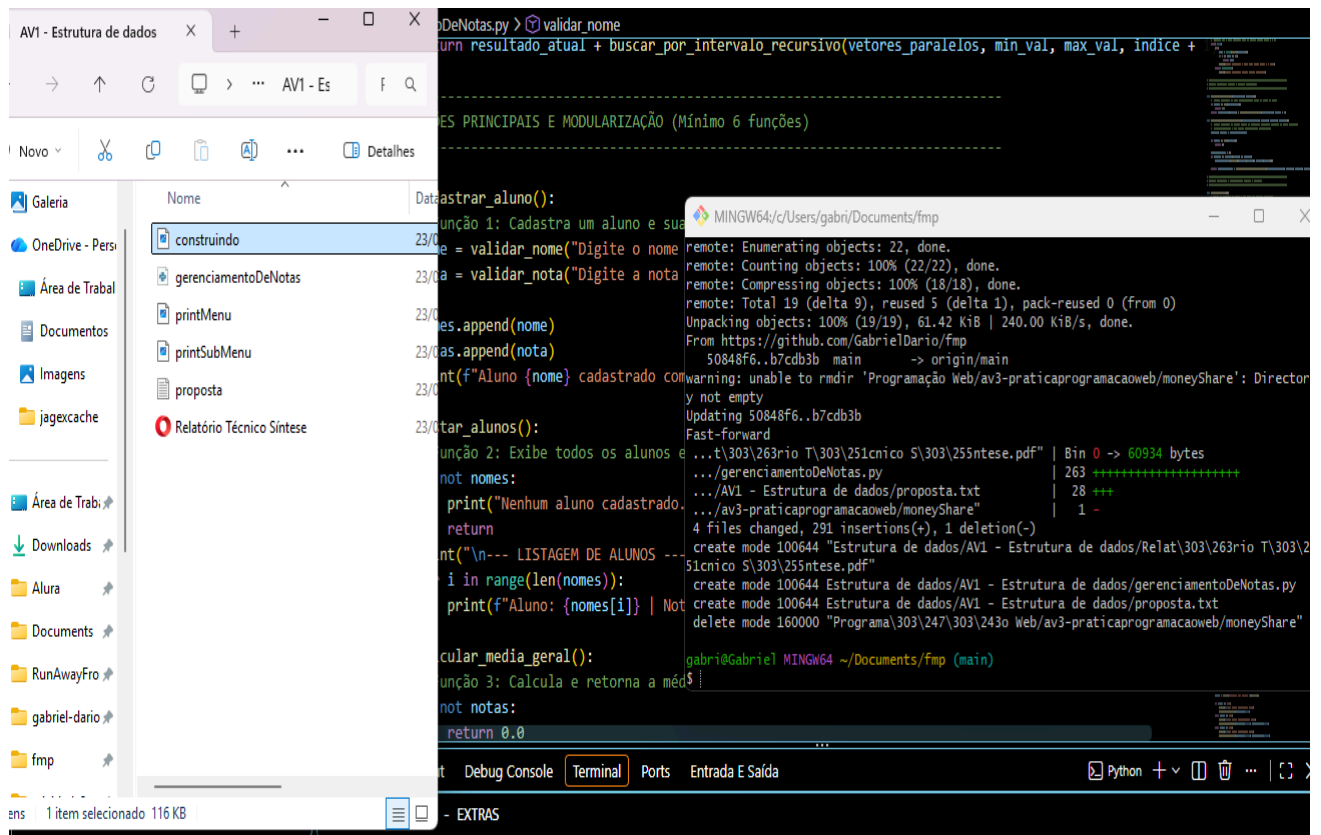
Capturas de tela



The screenshot displays a code editor interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'gerenciamentoDeNotas.py' with several files: 'AV1 - Estrutura de dados', 'gerenciamentoDeNotas.py', 'printMenu.png', 'printSubMenu.png', 'proposta.txt', and 'Relatório Técnico Síntese.pdf'. The code editor shows the 'gerenciamentoDeNotas.py' file with the following Python code:

```
1 # =====  
2 # SISTEMA DE GERENCIAMENTO DE NOTAS DE ALUNOS  
3 # Estrutura utilizando Vetores Paralelos (listas independentes para nomes e notas)  
4 # =====  
5  
6 # Vetores paralelos para armazenamento dos dados  
7 nomes = []  
8 notas = []  
9  
10 # -----  
11 # FUNÇÕES DE VALIDAÇÃO (Requisitos Não Funcionais)  
12 # -----  
13  
14 def validar_nome(mensagem):  
15     # Garante que o nome contenha apenas letras e espaços e não seja vazio  
16     while True:  
17         entrada = input(mensagem).strip()  
18         # Substitui espaços para verificar se o restante é alfabético  
19         if entrada.replace(" ", "").isalpha() and len(entrada) > 0:  
20             return entrada  
21         print("Opção inválida: O nome deve conter apenas letras.")  
22  
23 def validar_nota(mensagem):  
24     # Garante que a nota digitada seja um número válido entre 0 e 10  
25     while True:  
26         try:  
27             nota = float(input(mensagem))  
28             if 0 <= nota <= 10:  
29                 return nota  
30         except ValueError:  
31             print("Opção inválida: A nota deve ser um número entre 0 e 10.")  
32  
33 # =====  
34 # PROGRAMA PRINCIPAL  
35 # =====  
36  
37 # Inicialização  
38 printMenu()  
39 printSubMenu()  
40  
41 # Loop principal  
42 while True:  
43     print("\n===== MENU =====")  
44     print("1 - Cadastrar aluno")  
45     print("2 - Consultar aluno")  
46     print("3 - Consultar notas")  
47     print("4 - Calcular média")  
48     print("5 - Sair")  
49     print("=====")  
50     opcao = input("Escolha uma opção: ")  
51     if opcao == "1":  
52         nome = validar_nome("Digite o nome do aluno: ")  
53         nota = validar_nota("Digite a nota do aluno: ")  
54         # Adicionar aluno e nota aos vetores  
55         nomes.append(nome)  
56         notas.append(nota)  
57     elif opcao == "2":  
58         # Consultar aluno  
59     elif opcao == "3":  
60         # Consultar notas  
61     elif opcao == "4":  
62         # Calcular média  
63     elif opcao == "5":  
64         # Sair  
65         break  
66  
67 # Fim do programa  
68 print("Fim do programa.")
```

The code editor also shows a 'Terminal' tab at the bottom, which is currently empty. The status bar at the bottom indicates 'SUBMENU - EXTRAS'.



Explorer

gerenciamentoDeNotas.py

validar_nome

Open Editors

gerenciamentoDeNotas.py

AVI - Estrutura de dados

construindo.png

gerenciamentoDeNotas.py

printMenu.png

printSubMenu.png

proposta.txt

Relatório Técnico Síntese.pdf

Outline

Timeline

gerenciamentoDeNotas.py

def submenu():

else:

print("Opção inválida")

def menu_principal():

Gerenciador do Menu Principal

while True:

print("\n" + "="*30)

print(" GERENCIADOR DE NOTAS ")

print("="*30)

print("1 - Cadastrar alunos e notas")

print("2 - Listagem alunos com notas")

print("3 - Calcular e exibir média geral da turma")

print("4 - Maior e menor nota registrada")

print("5 - Mostrar alunos aprovados (média >= 7)")

print("6 - Abrir submenu")

print("0 - Encerrar aplicação")

opcao = input("Escolha uma opção: ").strip()

if opcao == '1':

cadastro_aluno()

elif opcao == '2':

listar_alunos()

elif opcao == '3':

if not notas:

print("Nenhum aluno cadastrado.")

else:

media = calcular_media_geral()

print(f"\nMédia geral da turma: {media:.2f}")

elif opcao == '4':

exibir_maior_menor_nota()

Teste de mesa

Menu principal :

```
=====
GERENCIADOR DE NOTAS
=====
1 - Cadastrar alunos e notas
2 - Listagem alunos com notas
3 - Calcular e exibir média geral da turma
4 - Maior e menor nota registrada
5 - Mostrar alunos aprovados (média >= 7)
6 - Abrir submenu
0 - Encerrar aplicação
Escolha uma opção: 1
Digite o nome do aluno: Joao
Digite a nota do aluno (0 a 10): 10
Aluno Joao cadastrado com sucesso!

Escolha uma opção: 2

--- LISTAGEM DE ALUNOS ---
Aluno: Joao | Nota: 10.00
Aluno: Pedro | Nota: 5.00

Escolha uma opção: 3

Média geral da turma: 7.50

=====
GERENCIADOR DE NOTAS
=====
1 - Cadastrar alunos e notas
2 - Listagem alunos com notas
3 - Calcular e exibir média geral da turma
4 - Maior e menor nota registrada
5 - Mostrar alunos aprovados (média >= 7)
6 - Abrir submenu
0 - Encerrar aplicação
Escolha uma opção: 1
Digite o nome do aluno: Pedro
Digite a nota do aluno (0 a 10): 5
Aluno Pedro cadastrado com sucesso!

Escolha uma opção: 4

Maior nota registrada: 10.00
Menor nota registrada: 5.00

Escolha uma opção: 5

--- ALUNOS APROVADOS (Média >= 7.0) ---
Aluno: Joao | Nota: 10.00
```

Submenu :

```
=====
SUBMENU - EXTRAS
=====
1 - Mostrar turma em ordem alfabética
2 - Mostrar turma em ordem decrescente de notas
3 - Mostrar turma em ordem crescente de notas
4 - Mostrar alunos reprovados (nota < 7)
5 - Listar alunos com notas de X a Y
6 - Desvio de nota individual em relação à média da turma
0 - Voltar ao menu principal
Escolha uma opção: 1
Escolha uma opção: 1
--- NOTAS CRESCENTES ---
Aluno: Pedro | Nota: 5.00
Aluno: Joao | Nota: 10.00
Escolha uma opção: 4
--- ALUNOS REPROVADOS (Nota < 7.0)
Aluno: Pedro | Nota: 5.00
Escolha uma opção: 2
--- NOTAS DECRESCENTES ---
Aluno: Joao | Nota: 10.00
Aluno: Pedro | Nota: 5.00
Escolha uma opção: 5
Digite o valor mínimo (X): 5
Digite o valor máximo (Y): 7
--- ALUNOS COM NOTAS ENTRE 5.00 E 7.00 ---
Aluno: Pedro | Nota: 5.00
Escolha uma opção: 6
Média atual da turma: 7.50
--- LISTAGEM DE ALUNOS ---
Aluno: Joao | Nota: 10.00
Aluno: Pedro | Nota: 5.00
Digite o nome exato do aluno para ver o desvio: Joao
Aluno: Joao
Nota do Aluno: 10.00
Desvio em relação à média (7.50): +2.50
```

Explicação do recurso:



Principais Aprendizados e Conceitos Aplicados

1. Estruturação e Manipulação de Dados (Vetores Paralelos)

A utilização de vetores paralelos para correlacionar o nome do aluno ao seu respectivo conjunto de notas permitiu gerenciar dados estruturados sem a necessidade inicial de objetos complexos, reforçando a lógica de manipulação de índices e sincronização de dados entre arrays.

2. Modularização e Limpeza de Código

Com a implementação de mais de 6 funções, o código foi dividido em responsabilidades bem definidas. A modularização facilita a reutilização de trechos repetitivos (como a listagem e validações de dados) e tornou a manutenção do código significativamente mais simples.

3. Pensamento Recursivo (Recursividade)

A implementação de pelo menos 2 funções recursivas (como a busca sequencial/filtragem recursiva e o cálculo somatório para a média) exigiu uma mudança de mentalidade em relação aos laços tradicionais (**for/while**). O entendimento de caso base e passo recursivo foi fundamental para evitar estouro de pilha (*stack overflow*) e resolver problemas dividindo-os em subproblemas menores.

4. Algoritmos de Ordenação e Filtragem

A criação das rotinas de ordenação alfabética e por nota (crescente/decrescente) permitiu compreender na prática como algoritmos de ordenação trabalham com trocas de posição simultâneas em vetores paralelos para manter a integridade dos dados.

5. Comparação Inter-linguagens (C vs. Python)

Desenvolver a mesma lógica em duas linguagens distintas trouxe *insights* valiosos:

- Em C: Exigiu controle manual de memória, manipulação detalhada de ponteiros/arrays de strings e tratamento estrito de tipos de dados.
- Em Python: Destacou a legibilidade, facilidade de manipulação de listas e estruturas de controle de fluxo mais intuitivas.